LSTM Autoencoders for Temporal Anomaly Detection

Lecture Note — Module 2, Block 3

MS Data Engineering

Contents

Table 1: *

	Symbol	Meaning
Notation	$\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L)$	Multivariate time series of length L
	$\boldsymbol{x}_t \in \mathbb{R}^d$	Observation at time t (d channels)
	T	Sliding window size
	s	Sliding window step size
	$\boldsymbol{W}_k$	k-th sliding window
	$\boldsymbol{h}_t, \boldsymbol{c}_t$	LSTM hidden state and cell state at time t
	d_h	Hidden dimension of the LSTM
	$\boldsymbol{f}_t, \boldsymbol{i}_t, \boldsymbol{o}_t$	Forget, input, and output gates
	σ_g	Sigmoid activation function
	$\odot$	Element-wise (Hadamard) product
	$s(\boldsymbol{X})$	Window-level anomaly score
	τ	Anomaly threshold

1 From Static to Temporal Anomaly Detection

The autoencoders studied in previous lectures treat each observation *independently*: an input vector $\boldsymbol{x} \in \mathbb{R}^d$ is compressed and reconstructed without reference to any other observation. Time series data, however, possesses intrinsic *temporal structure* that a vanilla autoencoder ignores. This section formalises the types of anomalies that arise in temporal data and motivates the need for sequence-aware architectures.

Definition 1.1: Time Series

A **multivariate time series** is an ordered sequence $\boldsymbol{X} = (\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_L)$ where each $\boldsymbol{x}_t \in \mathbb{R}^d$ is a d-dimensional observation recorded at time t. The integer L denotes the total length of the series and d the number of channels (or features).

1.1 Taxonomy of Temporal Anomalies

Definition 1.2: Point Anomaly

A **point anomaly** is a single observation $\boldsymbol{x}_t$ whose value is extreme or unexpected regardless of its temporal context. Formally, $\boldsymbol{x}_t$ is a point anomaly if its marginal likelihood under the data-generating process is very low:

$$p(\boldsymbol{x}_t) < \epsilon$$

for some small threshold $\epsilon > 0$. Example: a sudden large spike in a sensor reading.

Definition 1.3: Collective Anomaly

A **collective anomaly** is a contiguous subsequence $(\boldsymbol{x}_{t_0}, \boldsymbol{x}_{t_0+1}, \ldots, \boldsymbol{x}_{t_0+k})$ that is anomalous as a group, even though each individual point may appear normal in isolation. Formally, the joint probability of the subsequence is anomalously low:

$$p(\boldsymbol{x}_{t_0}, \ldots, \boldsymbol{x}_{t_0+k}) < \epsilon,$$

while the marginals $p(\boldsymbol{x}_t)$ may all be within normal range. Example: a sustained shift in the mean of a normally stationary process.

Definition 1.4: Contextual Anomaly

A **contextual anomaly** is an observation $\boldsymbol{x}_t$ that is normal in magnitude but anomalous *given its temporal context*. Formally, the conditional probability is anomalously low:

$$p(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}, \boldsymbol{x}_{t-2}, \dots) < \epsilon,$$

even though $p(\boldsymbol{x}_t)$ may be typical. Example: a value that would be expected at noon but occurs at 3 AM in a process with strong diurnal periodicity.

Remark 1.1: Limitations of static models

A vanilla (tabular) autoencoder computes $s(\boldsymbol{x}_t) = \|\boldsymbol{x}_t - \hat{x}_t\|^2$ independently for each t. It can detect point anomalies (large deviations from the learned marginal manifold) but *cannot* detect collective or contextual anomalies, since it encodes no notion of temporal order or conditional dependence. This is the fundamental motivation for sequence-to-sequence architectures.

2 Recurrent Neural Networks and LSTMs

Before presenting the LSTM autoencoder, we briefly review the recurrent architectures on which it is built.

2.1 Vanilla RNN

Definition 2.1: Recurrent Neural Network (RNN)

A **recurrent neural network** processes a sequence $(\boldsymbol{x}_1, \dots, \boldsymbol{x}_T)$ by maintaining a hidden state $\boldsymbol{h}_t \in \mathbb{R}^{d_h}$ that is updated at each time step:

$$\boldsymbol{h}_t = \sigma(\boldsymbol{W}_h \boldsymbol{h}_{t-1} + \boldsymbol{W}_x \boldsymbol{x}_t + \boldsymbol{b}), \tag{1}$$

where $\boldsymbol{W}_h \in \mathbb{R}^{d_h \times d_h}$, $\boldsymbol{W}_x \in \mathbb{R}^{d_h \times d}$, $\boldsymbol{b} \in \mathbb{R}^{d_h}$ are learnable parameters and σ is a nonlinear activation (typically tanh). The hidden state $\boldsymbol{h}_t$ serves as a compressed summary of the sequence up to time t.

Remark 2.1: Vanishing gradient problem

In a vanilla RNN, gradients of the loss with respect to early hidden states involve products of Jacobians $\prod_{k=t}^{T} \frac{\partial \boldsymbol{h}_k}{\partial \boldsymbol{h}_{k-1}}$. When T is large, these products tend to either vanish (if the spectral radius of the Jacobian is less than 1) or explode (if greater than 1). This makes it difficult for vanilla RNNs to learn long-range dependencies, a limitation directly addressed by gated architectures such as LSTM.

2.2 Long Short-Term Memory (LSTM)

Definition 2.2: LSTM Cell

A **Long Short-Term Memory (LSTM)** cell [1] augments the RNN with a **cell state** $\boldsymbol{c}_t \in \mathbb{R}^{d_h}$ and three **gating mechanisms** that control information flow. At each time step t,

given input $\boldsymbol{x}_t$ and previous states $(\boldsymbol{h}_{t-1}, \boldsymbol{c}_{t-1})$, the LSTM computes:

$$\boldsymbol{f}_t = \sigma_g(\boldsymbol{W}_f[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_f) \quad \text{(forget gate)}, \tag{2}$$
$$\boldsymbol{i}_t = \sigma_g(\boldsymbol{W}_i[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_i) \quad \text{(input gate)}, \tag{3}$$
$$\tilde{\boldsymbol{c}}_t = \tanh(\boldsymbol{W}_c[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_c) \quad \text{(candidate cell state)}, \tag{4}$$
$$\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t \quad \text{(cell state update)}, \tag{5}$$
$$\boldsymbol{o}_t = \sigma_g(\boldsymbol{W}_o[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_o) \quad \text{(output gate)}, \tag{6}$$
$$\boldsymbol{h}_t = \boldsymbol{o}_t \odot \tanh(\boldsymbol{c}_t) \quad \text{(hidden state)}, \tag{7}$$

where σ_g is the sigmoid function, $\odot$ denotes element-wise multiplication, and $[\cdot\,;\cdot]$ denotes concatenation.

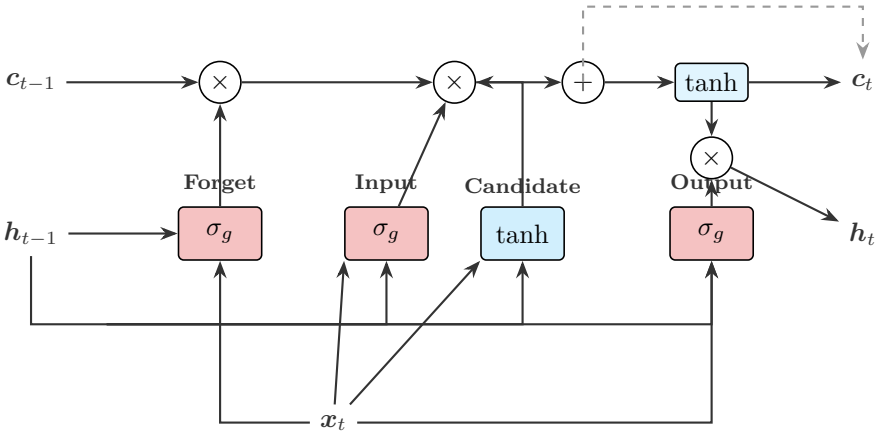

Figure 1: Internal structure of an LSTM cell. The three gates (forget, input, output) control the flow of information through the cell state $\boldsymbol{c}_t$. The additive cell state update is key to alleviating vanishing gradients.

Proposition 2.1: LSTM alleviates vanishing gradients

The cell state update in Eq. (5) is an *additive* interaction: $\boldsymbol{c}_t = \boldsymbol{f}_t \odot \boldsymbol{c}_{t-1} + \boldsymbol{i}_t \odot \tilde{\boldsymbol{c}}_t$. The gradient of the loss with respect to $\boldsymbol{c}_{t'}$ (for $t' < t$) contains a factor $\prod_{k=t'+1}^{t} \boldsymbol{f}_k$, which, when the forget gates are close to 1, allows gradients to flow across many time steps without vanishing. This is in contrast with the multiplicative Jacobian products in vanilla RNNs.

3 The LSTM Autoencoder Architecture

The LSTM Autoencoder (LSTM-AE), introduced by Malhotra et al. [2], is a sequence-to-sequence autoencoder where both the encoder and decoder are LSTM networks.

Definition 3.1: LSTM Autoencoder

An **LSTM Autoencoder** for sequences of length T with d features consists of:

1. An **encoder LSTM** that reads the input sequence $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$ and produces a final hidden state $\boldsymbol{h}_T \in \mathbb{R}^{d_h}$, which serves as the **bottleneck** (compressed representation of the entire sequence).
2. A **decoder LSTM** that takes $\boldsymbol{h}_T$ as its initial state and autoregressively produces a reconstructed sequence $(\hat{x}_1, \ldots, \hat{x}_T)$.

The model is trained to minimise the sequence-level reconstruction loss on windows of normal

data only.

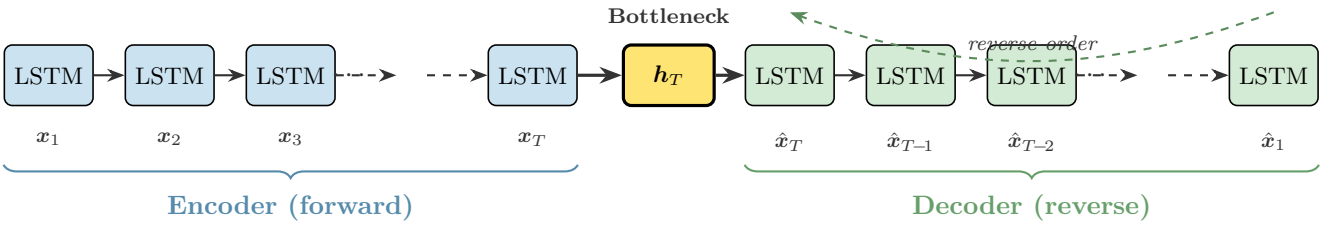

Figure 2: Architecture of the LSTM Autoencoder. The encoder reads the input sequence forward and compresses it into the bottleneck $\boldsymbol{h}_T$. The decoder reconstructs the sequence in **reverse order**, forcing the bottleneck to encode global temporal information.

3.1 Encoder

The encoder processes the input sequence step by step:

$$\boldsymbol{h}_t^{(\text{enc})}, \boldsymbol{c}_t^{(\text{enc})} = \text{LSTM}_{\text{enc}}(\boldsymbol{x}_t, \boldsymbol{h}_{t-1}^{(\text{enc})}, \boldsymbol{c}_{t-1}^{(\text{enc})}), \qquad t = 1, \ldots, T. \tag{8}$$

The bottleneck representation is $\boldsymbol{z} = \boldsymbol{h}_T^{(\text{enc})}$ (the hidden state after processing the entire input). With a multi-layer LSTM of L layers, $\boldsymbol{z}$ is the collection of hidden and cell states from the final time step across all layers.

3.2 Decoder

The decoder receives $\boldsymbol{z}$ as its initial hidden state and generates the reconstruction. Two common strategies exist for feeding inputs to the decoder:

(a) **Repeated hidden state:** At each decoding step, the same $\boldsymbol{z}$ is used as input:

$$\boldsymbol{h}_t^{(\text{dec})}, \boldsymbol{c}_t^{(\text{dec})} = \text{LSTM}_{\text{dec}}(\boldsymbol{z}, \boldsymbol{h}_{t-1}^{(\text{dec})}, \boldsymbol{c}_{t-1}^{(\text{dec})}), \qquad \hat{x}_t = \boldsymbol{W}_{\text{out}}\, \boldsymbol{h}_t^{(\text{dec})} + \boldsymbol{b}_{\text{out}}. \tag{9}$$

(b) **Autoregressive:** Each decoding step receives the previous output:

$$\boldsymbol{h}_t^{(\text{dec})}, \boldsymbol{c}_t^{(\text{dec})} = \text{LSTM}_{\text{dec}}(\hat{x}_{t-1}, \boldsymbol{h}_{t-1}^{(\text{dec})}, \boldsymbol{c}_{t-1}^{(\text{dec})}). \tag{10}$$

Remark 3.1: Reverse reconstruction order

A common and effective design choice is to reconstruct the sequence in **reverse order**, i.e. the decoder outputs $(\hat{x}_T, \hat{x}_{T-1}, \ldots, \hat{x}_1)$. The intuition is that the encoder's final hidden state $\boldsymbol{h}_T$ is temporally closest to $\boldsymbol{x}_T$; by decoding in reverse, the decoder first reconstructs the most recently seen input and must progressively "unwind" the entire sequence. This forces the bottleneck to encode global information about the full window, rather than relying on short-term recency effects. Empirically, reverse decoding often improves anomaly detection performance [2].

3.3 Anomaly Score

Definition 3.2: Window-Level Anomaly Score

For an input window $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T) \in \mathbb{R}^{T \times d}$ and its reconstruction $\hat{X} = (\hat{x}_1, \ldots, \hat{x}_T)$, the **window-level anomaly score** is:

$$s(\boldsymbol{X}) = \frac{1}{T} \sum_{t=1}^{T} \|\boldsymbol{x}_t - \hat{x}_t\|^2. \tag{11}$$

A window is flagged as anomalous if $s(\boldsymbol{X}) > \tau$.

Definition 3.3: Per-Feature Anomaly Score

For root cause analysis in multivariate time series, the **per-feature (per-channel) anomaly score** decomposes the window score across dimensions:

$$s_j(\boldsymbol{X}) = \frac{1}{T} \sum_{t=1}^{T} (x_{t,j} - \hat{x}_{t,j})^2, \qquad j = 1, \ldots, d. \tag{12}$$

This allows identification of which sensor or channel is primarily responsible for a detected anomaly.

4 Sliding Window Construction

The LSTM-AE operates on fixed-length windows extracted from the time series using a sliding window procedure.

Definition 4.1: Sliding Window Extraction

Given a time series $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_L)$, a **sliding window** of size T and step size s produces a set of overlapping subsequences:

$$\boldsymbol{W}_k = (\boldsymbol{x}_{1+(k-1)s},\ \boldsymbol{x}_{2+(k-1)s},\ \ldots,\ \boldsymbol{x}_{T+(k-1)s}), \qquad k = 1, 2, \ldots, \left\lfloor \frac{L-T}{s} \right\rfloor + 1.$$

Each window $\boldsymbol{W}_k \in \mathbb{R}^{T \times d}$ constitutes one training or test sample.

Remark 4.1: Window labelling convention

In anomaly detection, a window is typically labelled as **anomalous** if *any* time step within it is anomalous. This is a conservative (high-recall) convention:

$$y_k = \begin{cases} 1 & \text{if } \exists\, t \in [1 + (k-1)s,\ T + (k-1)s] : \text{label}(t) = \text{anomaly}, \\ 0 & \text{otherwise.} \end{cases}$$

The training set contains only windows with $y_k = 0$ (pure normal data).

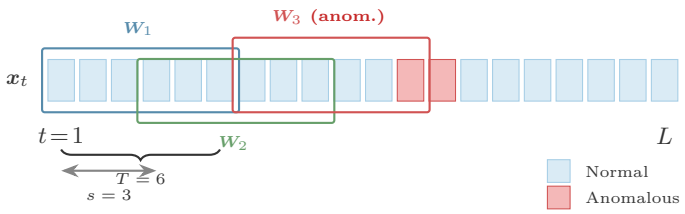

Figure 3: Sliding window extraction from a time series. Windows of size T are extracted with step s. A window is labelled anomalous if it contains any anomalous time step (red cells).

Theorem 4.1: Pointwise Score Coverage

For a time series of length L with sliding windows of size T and step $s = 1$, each interior time step t (with $T \leq t \leq L - T + 1$) is covered by exactly T overlapping windows. Boundary time steps ($t < T$ or $t > L - T + 1$) are covered by fewer windows. The pointwise score (Eq. 13) at interior points is therefore an average over T window scores, providing a smoothing effect with effective kernel width T.

Proof. Window $\boldsymbol{W}_k$ (with $s = 1$) contains time steps $\{k, k+1, \ldots, k+T-1\}$. Time step t belongs to window k if and only if $k \leq t \leq k + T - 1$, i.e. $t - T + 1 \leq k \leq t$. For interior points where both bounds lie in $[1, L - T + 1]$, this gives exactly T valid windows. The pointwise score is thus a uniform moving average of window scores with kernel width T. □

4.1 Pointwise Score from Window Scores

To obtain a per-timestep anomaly score from window-level scores, each time step t is assigned the score of the window centred on it (or the average score of all windows containing it when using step $s = 1$):

$$s_{\text{point}}(t) = \frac{1}{|\mathcal{K}_t|} \sum_{k \in \mathcal{K}_t} s(\boldsymbol{W}_k), \tag{13}$$

where $\mathcal{K}_t = \{k : t \in \text{window } k\}$ is the set of all windows containing time step t.

5 Threshold Selection for Temporal Anomaly Detection

The threshold strategies introduced for tabular autoencoders carry over to the temporal setting, with the score distribution now computed over *windows* rather than individual observations.

5.1 Unsupervised Strategies

(i) Training percentile: $\tau = Q_p(\{s(\boldsymbol{W}_k)\}_{k:y_k=0})$ for $p \in \{95, 97, 99\}$.

(ii) Mean + $k\sigma$: $\tau = \mu_{\text{train}} + k \cdot \sigma_{\text{train}}$ with $k \in \{2, 3\}$.

(iii) Extreme Value Theory: fit a Generalised Pareto Distribution to the upper tail of training window scores.

5.2 Supervised Strategy (Oracle)

When ground-truth labels are available for a validation set, the threshold that maximises the F_1 score can be found by grid search. This provides an *upper bound* on achievable performance and is useful as a benchmark, not as a deployable strategy.

Remark 5.1: Non-stationarity caveat

If the time series is non-stationary (e.g. has trends or evolving seasonality), a global threshold may be inappropriate. Adaptive thresholding methods, such as computing τ on a rolling window of recent training scores, or applying instance normalisation within each window before scoring, can improve robustness.

6 Effect of Window Size

The window size T is a critical hyperparameter that controls the temporal receptive field of the model.

Proposition 6.1: Window Size Trade-Off

The choice of window size T involves a trade-off between temporal coverage and granularity:

1. **Large** T: captures long-range temporal dependencies; improves detection of collective and contextual anomalies; increases computational cost and may "dilute" the score for short point anomalies (the anomalous time steps form a small fraction of the window).
2. **Small** T: faster training and inference; better sensitivity to short point anomalies; may miss long-range patterns such as gradual drift or periodic context violations.

Remark 6.1: Practical selection of T

In practice, T should be chosen to cover at least one full period of any known periodicity in the data (e.g. one diurnal cycle for sensor data sampled every minute would require $T \geq 1440$). When the anomaly types are unknown, a sweep over $T \in \{25, 50, 100, 200\}$ with cross-validated AUC-ROC is recommended.

Theorem 6.1: Score Dilution for Point Anomalies

Consider a window $\boldsymbol{X} = (\boldsymbol{x}_1, \ldots, \boldsymbol{x}_T)$ containing a single point anomaly at time t^* with excess reconstruction error $\Delta > 0$. Assume all other time steps achieve baseline reconstruction error $\bar{e}$. Then the window-level anomaly score is:

$$s(\boldsymbol{X}) = \bar{e} + \frac{\Delta}{T}. \tag{14}$$

As $T \to \infty$, the contribution of the anomaly vanishes: $s(\boldsymbol{X}) \to \bar{e}$. More precisely, for the anomaly to be detectable, we require $\Delta/T > \tau - \bar{e}$, i.e. the anomaly magnitude Δ must grow linearly with window size T to remain above threshold.

Proof. By Definition 3.2, $s(\boldsymbol{X}) = \frac{1}{T}\sum_{t=1}^{T} \|\boldsymbol{x}_t - \hat{x}_t\|^2 = \frac{1}{T}((T-1)\bar{e} + (\bar{e} + \Delta)) = \bar{e} + \frac{\Delta}{T}$. □

Remark 6.2: Mitigating dilution

This dilution effect can be mitigated by using the **maximum** rather than the mean over the window: $s_{\max}(\boldsymbol{X}) = \max_{1 \leq t \leq T} \|\boldsymbol{x}_t - \hat{x}_t\|^2$. Alternatively, per-timestep scoring via Eq. (13) with small step size effectively decomposes the window score and localises the anomaly.

Proposition 6.2: LSTM Bottleneck Information Capacity

The encoder LSTM compresses a window of $T \cdot d$ real values into a bottleneck of dimension d_h. The **compression ratio** is:

$$\rho = \frac{T \cdot d}{d_h}.$$

For effective anomaly detection, we need $\rho > 1$ (the bottleneck is a genuine constraint). However, the effective capacity of the LSTM also depends on the number of layers L: a multi-layer LSTM stores (h, c) pairs for each layer, yielding a total bottleneck dimensionality of $2Ld_h$. The effective compression ratio is therefore:

$$\rho_{\text{eff}} = \frac{T \cdot d}{2L \cdot d_h}.$$

If $\rho_{\text{eff}} \leq 1$, the bottleneck is not constraining and the model may learn to perfectly reconstruct anomalies.

7 Limitations and When LSTM-AE Fails

L1. Fixed window length. The LSTM-AE requires a fixed T, which may not align with the natural scale of anomalies. Anomalies shorter or longer than the window may be poorly captured.

L2. Long-range dependencies beyond the window. Even within a window, LSTMs struggle with very long sequences (hundreds to thousands of steps). Transformer-based autoencoders (which use self-attention) can capture longer-range dependencies more effectively.

L3. Non-stationarity. If the data distribution shifts over time (concept drift), a model trained on historical data may produce false positives on new but normal patterns. Continual or online learning approaches are required.

L4. Training data contamination. If anomalous windows contaminate the training set, the LSTM-AE may learn to reconstruct anomalous patterns, reducing discrimination. Robust training strategies (e.g. iterative outlier filtering) can mitigate this.

L5. Multivariate heterogeneity. When different channels have vastly different scales or dynamics, per-channel normalisation and separate score aggregation are necessary to avoid one channel dominating the anomaly score.

8 Extensions and Related Methods

8.1 Bidirectional LSTM Encoder

Definition 8.1: Bidirectional LSTM

A **bidirectional LSTM** (BiLSTM) processes the input sequence in both forward and backward directions, producing two hidden state sequences:

$$\overrightarrow{\boldsymbol{h}}_t = \text{LSTM}_{\text{fwd}}(\boldsymbol{x}_t, \overrightarrow{\boldsymbol{h}}_{t-1}), \qquad t = 1, \ldots, T, \tag{15}$$

$$\overleftarrow{\boldsymbol{h}}_t = \text{LSTM}_{\text{bwd}}(\boldsymbol{x}_t, \overleftarrow{\boldsymbol{h}}_{t+1}), \qquad t = T, \ldots, 1. \tag{16}$$

The combined representation at each time step is $\boldsymbol{h}_t = [\overrightarrow{\boldsymbol{h}}_t; \overleftarrow{\boldsymbol{h}}_t] \in \mathbb{R}^{2d_h}$, and the bottleneck is $\boldsymbol{z} = [\overrightarrow{\boldsymbol{h}}_T; \overleftarrow{\boldsymbol{h}}_1] \in \mathbb{R}^{2d_h}$.

Proposition 8.1: BiLSTM advantage for window-based encoding

In a unidirectional encoder, the hidden state $\boldsymbol{h}_T$ has direct access to recent inputs $(\boldsymbol{x}_T, \boldsymbol{x}_{T-1}, \ldots)$ but must compress early inputs $(\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots)$ through many recurrent steps, incurring potential information loss. The BiLSTM bottleneck $[\overrightarrow{\boldsymbol{h}}_T; \overleftarrow{\boldsymbol{h}}_1]$ has direct access to

both ends of the sequence, providing a more balanced representation. However, this doubles the bottleneck dimensionality to $2d_h$, which may require reducing d_h to maintain the compression constraint $2d_h < T \cdot d$.

8.2 Gated Recurrent Unit (GRU)

Definition 8.2: GRU Cell

The **Gated Recurrent Unit** [5] is a simplified gated recurrent architecture that merges the cell state and hidden state into a single vector $\boldsymbol{h}_t$, using two gates:

$$\boldsymbol{r}_t = \sigma_g(\boldsymbol{W}_r[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_r) \quad \text{(reset gate)}, \tag{17}$$

$$\boldsymbol{u}_t = \sigma_g(\boldsymbol{W}_u[\boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_u) \quad \text{(update gate)}, \tag{18}$$

$$\tilde{\boldsymbol{h}}_t = \tanh(\boldsymbol{W}_h[\boldsymbol{r}_t \odot \boldsymbol{h}_{t-1}; \boldsymbol{x}_t] + \boldsymbol{b}_h) \quad \text{(candidate)}, \tag{19}$$

$$\boldsymbol{h}_t = (1 - \boldsymbol{u}_t) \odot \boldsymbol{h}_{t-1} + \boldsymbol{u}_t \odot \tilde{\boldsymbol{h}}_t \quad \text{(state update)}. \tag{20}$$

Remark 8.1: GRU vs LSTM trade-off

The GRU has $\sim$25% fewer parameters than an LSTM of the same hidden dimension (2 gates instead of 3, no separate cell state). Empirically, GRUs perform comparably to LSTMs on many sequence modelling tasks while being faster to train. For anomaly detection, the choice between GRU-AE and LSTM-AE is typically made via cross-validation; neither dominates uniformly.

8.3 Other Extensions

- **OmniAnomaly** [3]: combines a VAE with a stochastic recurrent model (using normalising flows for the posterior) and a planar flow prior. The stochastic latent variable at each time step captures both temporal dynamics and uncertainty, providing richer anomaly scores than deterministic LSTMs.
- **USAD** [4]: an *adversarially* trained autoencoder for multivariate time series. Two decoders are trained in an adversarial manner: one to reconstruct, the other to "fool" the first, yielding a score that is sensitive to subtle deviations.
- **Transformer-AE:** replaces LSTM layers with Transformer encoder/decoder blocks. Self-attention allows the model to directly attend to any time step in the window, avoiding the sequential bottleneck of recurrence. Effective for very long windows.
- **Temporal Convolutional Network (TCN)-AE:** uses dilated causal convolutions instead of recurrence. TCNs offer parallelisable training and adjustable receptive fields via dilation factors.

9 Summary

Table 2 summarises the key properties of the LSTM-AE for temporal anomaly detection.

References

Table 2: LSTM Autoencoder: summary of properties.

Aspect	**LSTM-AE**
Input	Sequences of shape (T, d)
Bottleneck	Final hidden state $\boldsymbol{h}_T$ of encoder LSTM
Anomaly score	Window-level MSE (or per-feature decomposition)
Detects	Point, collective, and contextual anomalies
Key hyperparameter	Window size T, hidden dimension d_h
Limitation	Requires fixed window length; slow on very long sequences

References

[1] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, 9(8):1735–1780, 1997.

[2] P. Malhotra *et al.*, "LSTM-based encoder-decoder for multi-sensor anomaly detection," *ICML Anomaly Detection Workshop*, 2016.

[3] Y. Su *et al.*, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," *KDD*, 2019.

[4] J. Audibert *et al.*, "USAD: UnSupervised Anomaly Detection on multivariate time series," *KDD*, 2020.

[5] K. Cho *et al.*, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," *EMNLP*, 2014.